Backend/Full-Stack Lead Developer Assignment: Mini RFQ Portal

Oct 10, 2026 · @Mohit Verma

Overview

We are hiring a full-time Lead Developer at Bidveda who will own how our product is designed, built and shipped. This assignment is a small version of that job: understand a problem, make decisions, design it, build it, and explain it.

You will build a **Mini RFQ Portal**. We expect about **6 to 8 focused hours of work**, spread over **4 days**. Please do not polish endlessly. Deciding what to build, what to simplify and what to leave out is part of what we are evaluating.

You can use any tech stack you are comfortable with. Tell us why you chose it.

Questions: If anything is unclear, you can email us your questions before you start. You can also make a reasonable assumption and write it down in your design document. Both are fine. We care about how you handle uncertainty.

Problem statement and example

Build a web application where a **buyer** company publishes Requests for Quotation (RFQs), **vendors** submit quotes against them, and the buyer compares the quotes and awards the RFQ to one vendor.

Roles

Role	What they do
Buyer	Creates and publishes RFQs, compares quotes, awards one vendor
Vendor	Browses open RFQs, submits and revises quotes before the deadline
Admin	Manages users and can view everything

A simple example

Acme Constructions (a buyer) needs office supplies and publishes this RFQ:

- **Title:** Office supplies for Hyderabad site

- **Deadline:** 20 Nov, 5:00 PM
- **Delivery location:** Hyderabad
- **Line items:** 100 chairs, 20 desks

Two vendors respond before the deadline:

Vendor	Chairs (per unit)	Desks (per unit)	Total	Delivery
Vendor A	2,000	8,000	3,60,000	10 days
Vendor B	2,200	7,000	3,60,000	7 days

Vendor A later revises the chair price to 1,900 before the deadline, which brings its total down to 3,50,000. After the deadline passes, no one can submit or change a quote. The buyer opens the comparison view, sorts by total or delivery time, and awards the RFQ to Vendor B, choosing faster delivery over the slightly lower price. Vendor B's quote becomes Accepted, Vendor A's becomes Rejected, and the RFQ becomes Awarded.

How statuses move

- RFQ: Draft → Published → Closed → Awarded
- Quote: Submitted → Shortlisted → Accepted or Rejected

The example is only to explain the idea. You decide the exact data model, the screens and the API design.

Functional requirements

Must have

1. **Auth and roles:** Sign up and login for Buyer, Vendor and Admin, with role-based access.
2. **RFQ management (Buyer):** Create, edit and publish an RFQ with title, description, category, line items (name, quantity, unit), delivery location, optional budget and submission deadline.
3. **Quote submission (Vendor):** Submit a quote with a price per line item, delivery time and notes. A vendor can revise the quote until the deadline. No submissions or changes after it.
4. **Comparison view (Buyer):** Compare all quotes for an RFQ side by side, sortable by total price and delivery time.
5. **Award flow:** The buyer awards one quote. The RFQ and quote statuses update as described above, and the history of status changes is kept.
6. **File upload and download:** Buyers attach specification files to an RFQ. Vendors attach documents to a quote. PDF and XLSX are enough.

7. **Search and filter:** Vendors browse open RFQs by keyword, category, location and deadline, with pagination. Buyers filter their own RFQs by status.
8. **Frontend:** At minimum an RFQ list, an RFQ detail page, an RFQ create/edit form, a quote form and the comparison view.
9. **Database:** A relational database (PostgreSQL or MySQL) with migrations.
10. **Tests:** Automated tests for the key flows (see the code section).
11. **Deployment:** A live URL, or a Docker setup that runs with one command.

Optional bonus (only if time allows)

- **Audit log:** A record of who changed what and when.

A well-finished core is better than a half-finished core with a bonus.

What is left open on purpose

This document does not specify everything. A real requirement never does. Where something is unclear, decide, then write the decision and the reason in your design document under "Assumptions".

A couple of examples of things you will need to decide:

- Can a vendor see other vendors' quotes, or should they stay hidden?
- When a vendor revises a quote, what happens to the old version?

We are not looking for one "right" answer. We are looking for a sensible answer that is clearly stated and consistently applied.

Deliverable 1: System design document

Write a short design document (a Markdown file in the repo, a PDF or a Google Doc). We suggest writing it before you start coding, since it will guide your build. You will submit it together with the code and the video in a single submission. It should contain these sections:

#	Section	What we expect
1	Requirement understanding	The problem in your own words, the main user flows per role, what is in scope and what is out of scope
2	Assumptions and open questions	Every assumption you made where the requirement was unclear, with a one-line reason

#	Section	What we expect
3	Tech stack and reasoning	Frontend, backend, database, file storage, deployment. Why each, and what you considered instead
4	Architecture and services	The components needed (app, database, file storage, background jobs, cache, anything else) and how they talk to each other. A simple diagram is enough
5	Database model	An ER diagram or table definitions with keys, constraints and important indexes. Explain how you store line items, quote revisions, status history and money
6	API design	List of endpoints with method, path, role allowed, and a short purpose. Include request and response examples for the 3 most important ones
7	Key flows	Step by step for: submitting a quote before the deadline, rejecting a late quote, awarding an RFQ
8	Security and access control	How roles are enforced, how one vendor is prevented from reading another vendor's data, how file uploads are made safe
9	Test cases	The test cases you thought of for the main flows and edge cases. Mark which ones you implemented and which ones you would add with more time. This can be in this document or in a separate document
10	Scalability and next steps	What breaks first at 100x load, what you would change, and what you skipped on purpose

You may update the design document while coding. If the final code differs from your design, add a short note on what changed and why. That is normal and we like to see it.

Deliverable 2: Code in GitHub

Build the application and push it to a GitHub repository. Share the repository link with us as part of your submission.

What the repository should contain

- **Working frontend and backend** covering the must-have requirements

- **README** with setup steps (ideally one command to run), environment variables, a short architecture overview, and how to run the tests
- **The design document** from Deliverable 1, updated if the design changed. Your test cases can live in it or in a separate document
- **Database migrations** and a seed script with sample buyers, vendors and RFQs so we can try it quickly
- **API documentation** as Swagger/OpenAPI or a Postman collection
- **Tests** that you can run with a single command
- **Docker setup** (Dockerfile and docker-compose) or a live deployed URL, with test login credentials for each role
- `.env.example` with no real secrets committed

Testing expectations

- Cover the key flows, mainly through API-level tests.
- A few meaningful tests are better than many trivial ones.
- It is fine if some of your test cases are not automated. Mark them clearly in your test documentation.

Add a "Known limitations and next steps" section to the README. Be honest about what is incomplete, what you would fix first, and what you would do for scale.

Video, AI tools and submission checklist

Walkthrough video (5 to 10 minutes). Record a screen share (Loom or similar) that shows the app working and explains your main design decisions. This is a mandatory part of your submission.

AI tools. You may use AI coding assistants. Please add a short note in the README on where and how you used them. In the interview we will ask you to explain and change parts of your code, so you should understand everything you submit.

Submission checklist

- [ ] System design document (all 10 sections), including your test cases
- [ ] GitHub repository link
- [ ] README with setup steps, assumptions, and known limitations
- [ ] Working app: live URL or Docker setup, with test credentials for Buyer, Vendor and Admin
- [ ] API documentation
- [ ] Tests, with the command to run them
- [ ] Seed data

- [ ] Walkthrough video (5 to 10 minutes)
- [ ] Optional: audit log bonus, described in the README

Timeline

- You have **4 days** from the day you receive this document. If you have questions, please send them on the first day.
- Submit everything together in a single submission (design document, GitHub link and video) to **tech@bidveda.in**.

Thank you for investing your time. We are looking forward to seeing how you think.